Can you help me make the palyer and state data class

Absolutely! Let's build the `Player` and `State` dataclasses for your Basketball Tracker one step at a time.

Step 1: Import the tools

At the very top of your code, write:

```
from dataclasses import dataclass, field
```

- `dataclass` makes it easier to create classes that store information.
- `field` allows us to give certain variables default values.

Step 2: Create the Player dataclass

```
@dataclass
class Player:
    name: str
    jersey_number: int
    points: int
    rebounds: int
    assists: int
```

What does each variable mean?

Variable	Data type	Purpose
`name`	`str`	Player's name
`jersey_number`	`int`	Jersey number
`points`	`int`	Points scored
`rebounds`	`int`	Total rebounds
`assists`	`int`	Total assists

Step 3: Create the State dataclass

```
@dataclass
class State:
    players: list[Player] = field(default_factory=list)
    team_name: str = "My Basketball Team"
    games_played: int = 0
```

```
    selected_player: str = ""
```

What does each variable mean?

Variable	Data type	Purpose
`players`	`list[Player]`	Stores all players
`team_name`	`str`	Stores the team name
`games_played`	`int`	Tracks games played
`selected_player`	`str`	Stores the currently selected player

Step 4: Understand how they work together

Think of it this way:

- Player stores information about one basketball player.
- State stores information about the entire basketball team.

For example, your team might have 10 players. The `players` list in State stores all 10 Player objects.

Step 5: Your completed code

Put everything together:

```
from dataclasses import dataclass, field
from drafter import *

@dataclass
class Player:
    name: str
    jersey_number: int
    points: int
    rebounds: int
    assists: int

@dataclass
class State:
    players: list[Player] = field(default_factory=list)
```

```
    team_name: str = "My Basketball Team"
    games_played: int = 0
    selected_player: str = ""
```

First section complete

For your assignment explanation, you can write:

"The Player dataclass stores each basketball player's individual information, while the State dataclass stores the information that needs to be remembered throughout the website, including the roster, team name, games played, and selected player."

Next, we can work on the homepage (`index`) and connect it to your State dataclass.

Can you help me make the homepage of the code and the team roster

Yes! Let's build these two pages next. We'll do them separately so you understand what each part of the code does.

Step 1: Create the homepage (`index`)

The homepage is the first page users see when they open your Basketball Tracker.

Add this code underneath your `State` dataclass:

```
@route
def index(state: State) -> Page:
    """Displays the basketball tracker homepage."""
    return Page(state, [
        Text("BASKETBALL TRACKER"),
        Text("--------------------"),
        Text("Welcome to your basketball team dashboard!"),
        Text("Manage players and track their statistics."),
        Text("Team: " + state.team_name),
        Text("Games Played: " + str(state.games_played)),
        Text("Total Players: " + str(len(state.players)))
    ])
```

Understanding the code

- `@route` tells Drafter that this function creates a website page.

- `def index(state: State)` creates the homepage and receives the website's current information.
- `Page(state, [...])` creates the page.
- `Text()` displays words on the screen.
- `state.team_name` displays the team name.
- `state.games_played` displays the number of games.
- `len(state.players)` counts how many players are on the roster.
- `str()` converts numbers into text so they can be displayed.

Step 2: Create the team roster (`players`)

This page displays everyone added to the basketball team.

Add this code underneath the homepage:

```
@route
def players(state: State) -> Page:
    """Displays all players on the team."""
    content = [
        Text("TEAM ROSTER"),
        Text("--------------------")
    ]

    if len(state.players) == 0:
        content.append(Text("No players have been added yet."))
    else:
        for player in state.players:
            content.append(Text(
                player.name + " | Jersey #" +
                str(player.jersey_number)
            ))

            content.append(Text(
                "Points: " + str(player.points) +
                " | Rebounds: " + str(player.rebounds) +
                " | Assists: " + str(player.assists)
            ))
```

```
    content.append(Button("Home", "/"))

    return Page(state, content)
```

Understanding the code

There are three important concepts here:

1. If statement

```
if len(state.players) == 0:
```

Checks whether the team has any players.

2. Else statement

```
else:
```

If players exist, the program moves to the next section.

3. For loop

```
for player in state.players:
```

Goes through every player in the roster and displays their information.

Step 3: Test your code

After adding both functions, your website should have:

Homepage preview



BASKETBALL TRACKER

Welcome to your basketball team dashboard!

Team: My Basketball Team

Games Played: 0

Total Players: 0

Team roster preview

TEAM ROSTER

No players have been added yet.

The roster will display player information once players are added.

can you help me make the add player, player statistics, and update player

Absolutely! Let's build these three sections one at a time so you understand what each part does.

We'll use the `Player` and `State` dataclasses and the homepage and team roster code we already made.

1. Add Player

This page lets you enter a player's name, jersey number, points, rebounds, and assists.

Step 1: Add the page

```
@route
def add_player(state: State) -> Page:
    return Page(state, [
        Text("ADD A PLAYER"),
        Text("--------------------"),
        TextBox("name", "Player Name"),
        TextBox("jersey", "Jersey Number"),
        TextBox("points", "Points"),
        TextBox("rebounds", "Rebounds"),
        TextBox("assists", "Assists"),
        Button("Add Player", add_player_submit),
        Button("Home", "/")
    ])
```

Step 2: Add the function that saves the player

```
@route
def add_player_submit(state: State, name: str,
                      jersey: str, points: str,
                      rebounds: str, assists: str) -> Page:

    if name.strip() == "":
        return Page(state, [
            Text("Please enter a player name."),
            Button("Try Again", add_player)
        ])
```

```
    if not jersey.isdigit() or not points.isdigit():
        return Page(state, [
            Text("Jersey number and points must be numbers."),
            Button("Try Again", add_player)
        ])

    if not rebounds.isdigit() or not assists.isdigit():
        return Page(state, [
            Text("Rebounds and assists must be numbers."),
            Button("Try Again", add_player)
        ])

    new_player = Player(
        name,
        int(jersey),
        int(points),
        int(rebounds),
        int(assists)
    )

    state.players.append(new_player)

    return Page(state, [
        Text(name + " was successfully added!"),
        Button("View Players", players),
```

What this code does:

- `TextBox` creates places to enter information.
- `if` checks that the information is valid.
- `isdigit()` checks that the numbers contain only digits.
- `Player()` creates a new player.
- `append()` adds the player to the team roster.

2. Player Statistics

This page lets you select a player and see their statistics.

```
@route
```

```
def stats(state: State) -> Page:
    content = [
        Text("PLAYER STATISTICS"),
        Text("--------------------")
    ]

    if len(state.players) == 0:
        content.append(Text("No players available."))
    else:
        content.append(Text("Select a player:"))

        for player in state.players:
            content.append(
                Button(player.name, view_stats, player.name)
            )

    content.append(Button("Home", "/"))
    return Page(state, content)
```

Now add the function that displays the selected player's stats:

```
@route
def view_stats(state: State, name: str) -> Page:
    state.selected_player = name

    for player in state.players:
        if player.name == name:
            return Page(state, [
                Text("STATS FOR " + player.name.upper()),
                Text("--------------------"),
                Text("Jersey Number: " + str(player.jersey_number)),
                Text("Points: " + str(player.points)),
                Text("Rebounds: " + str(player.rebounds)),
                Text("Assists: " + str(player.assists)),
                Button("Back to Players", stats),
                Button("Home", "/")
            ])

    return Page(state, [
```

```
        Text("Player not found."),
        Button("Back to Statistics", stats),
        Button("Home", "/")
    ])
```

What this code does:

- The `for` loop goes through every player.
- Each player gets their own button.
- Clicking a player's button displays their statistics.
- `upper()` makes the player's name uppercase.

3. Update Player Statistics

This section lets you choose a player whose statistics you want to update.

Step 1: Create the update page.

```
@route
def update_player(state: State) -> Page:
    content = [
        Text("UPDATE PLAYER STATS"),
        Text("--------------------")
    ]

    if len(state.players) == 0:
        content.append(Text("No players available to update."))
    else:
        for player in state.players:
            content.append(
                Button(player.name, update_stats, player.name)
            )

    content.append(Button("Home", "/"))
    return Page(state, content)
```

Step 2: Create the function that displays the update form.

```
@route
def update_stats(state: State, name: str) -> Page:
```

```
    for player in state.players:
        if player.name == name:
            return Page(state, [
                Text("UPDATE " + player.name.upper()),
                TextBox("points", "New Points"),
                TextBox("rebounds", "New Rebounds"),
                TextBox("assists", "New Assists"),
                Button("Save Statistics", save_stats, name),
                Button("Home", "/")
            ])

    return Page(state, [
        Text("Player not found."),
        Button("Home", "/")
    ])
```

Step 3: Create the function that saves the updated statistics.

```
@route
def save_stats(state: State, name: str,
               points: str, rebounds: str,
               assists: str) -> Page:

    if not points.isdigit() or not rebounds.isdigit() or not assists.isdigit():
        return Page(state, [
            Text("Please enter numbers for all statistics."),
            Button("Try Again", update_player)
        ])

    for player in state.players:
        if player.name == name:
            player.points = int(points)
            player.rebounds = int(rebounds)
            player.assists = int(assists)

            return Page(state, [
                Text("Statistics successfully updated!"),
                Button("View Statistics", stats),
                Button("Home", "/")
```

```
            ])

    return Page(state, [
        Text("Player not found."),
        Button("Home", "/")
    ])
```

Important: Test these sections

I recommend adding the code in this order:

1. Add Player and Add Player Submit.
2. Player Statistics and View Stats.
3. Update Player, Update Stats, and Save Stats.

can you help me make the leaderboard, delete player, team settings , and record game

Absolutely! Let's finish the next four sections of your basketball tracker. We'll keep the code similar to the sections we already made.

1. Leaderboard

The leaderboard organizes players from the most points to the fewest points.

```
@route
def leaderboard(state: State) -> Page:
    content = [
        Text("TEAM LEADERBOARD"),
        Text("--------------------")
    ]

    if len(state.players) == 0:
        content.append(Text("No players available yet."))
    else:
        ranked_players = sorted(
            state.players,
            key=lambda player: player.points,
            reverse=True
        )

        position = 1
```

```
        for player in ranked_players:
            content.append(Text(
                str(position) + ". " + player.name +
                " - " + str(player.points) + " points"
            ))
            position += 1

    content.append(Button("Home", "/"))
    return Page(state, content)
```

Important things to understand:

- `sorted()` organizes the players.
- `key=lambda player: player.points` sorts by points.
- `reverse=True` puts the highest score first.
- `position` keeps track of each player's ranking.

2. Delete Player

This page allows you to choose a player to delete.

```
@route
def delete_player(state: State) -> Page:
    content = [
        Text("DELETE A PLAYER"),
        Text("--------------------")
    ]

    if len(state.players) == 0:
        content.append(Text("There are no players to delete."))
    else:
        for player in state.players:
            content.append(
                Button(player.name, confirm_delete, player.name)
            )

    content.append(Button("Home", "/"))
    return Page(state, content)
```

Now create a confirmation page:

```
@route
def confirm_delete(state: State, name: str) -> Page:
    state.selected_player = name

    return Page(state, [
        Text("DELETE " + name + "?"),
        Text("Are you sure you want to remove this player?"),
        Button("Yes, Delete Player", remove_player, name),
        Button("Cancel", delete_player),
        Button("Home", "/")
    ])
```

Finally, create the function that removes the player:

```
@route
def remove_player(state: State, name: str) -> Page:
    for player in state.players:
        if player.name == name:
            state.players.remove(player)
            state.selected_player = ""

            return Page(state, [
                Text(name + " was successfully deleted."),
                Button("View Players", players),
                Button("Home", "/")
            ])

    return Page(state, [
        Text("Player not found."),
        Button("Home", "/")
    ])
```

Important things to understand:

- `confirm_delete` asks the user to confirm.
- `remove_player` removes the selected player.
- `.remove(player)` deletes that player from the list.

- The Cancel button takes you back without deleting anyone.

3. Team Settings

This page allows you to change your team's name and see how many games have been played.

```
@route
def team_settings(state: State) -> Page:
    return Page(state, [
        Text("TEAM SETTINGS"),
        Text("--------------------"),
        Text("Current Team: " + state.team_name),
        TextBox("team_name", "New Team Name"),
        Button("Save Team Name", save_team_name),
        Text("Games Played: " + str(state.games_played)),
        Button("Record a Game", record_game),
        Button("Home", "/")
    ])
```

Now add the function that saves the new team name:

```
@route
def save_team_name(state: State, team_name: str) -> Page:
    if team_name.strip() == "":
        return Page(state, [
            Text("Please enter a team name."),
            Button("Try Again", team_settings)
        ])

    state.team_name = team_name

    return Page(state, [
        Text("Team name successfully changed!"),
        Text("New Team Name: " + state.team_name),
        Button("Team Settings", team_settings),
        Button("Home", "/")
    ])
```

Important things to understand:

- `state.team_name` stores the team's name.
- `TextBox` lets the user enter a new name.
- `.strip()` checks whether the user entered only spaces.
- The new name is saved in the State dataclass.

4. Record Game

This function increases the number of games played by one.

```
@route
def record_game(state: State) -> Page:
   state.games_played += 1

   return Page(state, [
       Text("Game successfully recorded!"),
       Text("Total Games Played: " + str(state.games_played)),
       Button("Team Settings", team_settings),
       Button("Home", "/")
   ])
```

Important things to understand:

- `+= 1` increases the game count by one.
- `state.games_played` stores the total number of games.

Final checklist

Once you've added everything, your basketball tracker should have:

Homepage
Team Roster
Add Player
Player Statistics
Update Player Statistics
Leaderboard
Delete Player
Team Settings
Record Game

One important reminder: Your `State` dataclass needs these fields:

```
players: list[Player] = field(default_factory=list)
team_name: str = "My Basketball Team"
games_played: int = 0
selected_player: str = ""
```